

Как языковые модели научились видеть?

**Екатерина Трофимова**

Автор статьи

**Денис Рябоконь**

Ревьюер статьи

Введение

BERT (Google), GPT-2, GPT-3 (OpenAI) [2018-2020 гг.]

Типы моделей

Предварительное обучение LLM

CLIP (OpenAI) [2021 г.]Flamingo (DeepMind) [2022 г.]

3D-LLM и пространственное мышление

3D-LLM [2023 г.]Uni3D [2023 г.]

Заключение

Введение

Сначала большие языковые модели (LLM) работали исключительно с текстом, не обрабатывая входные данные других модальностей: изображения, видео, аудио. Однако развитие архитектур и методов обучения позволило им стать полноценными мультимодальными системами.

Сегодня мы расскажем, как LLM эволюционировали в сложные мультимодальные системы, способные «видеть», а также разберём технические принципы их работы.

BERT (Google), GPT-2, GPT-3 (OpenAI) [2018-2020 гг.]

Еще с середины прошлого века людей интересовала задача машинного перевода. С появлением нейронных сетей её решение получило новый виток развития: от простых sequence-to-sequence (seq2seq) архитектур на основе рекуррентных сетей до механизма внимания.

В 2017 году появляется архитектура-**трансформер**, основа которой — attention block. Она предполагает отказ от рекуррентности, но следует идее seq2seq в виде энкодер-декодер архитектуры. Механизм внимания в архитектуре реализуется за счёт присвоения трём векторам полученным из исходных эмбеддингов токенов, ролей запросов (queries), ключей (keys) и значений (values). Энкодер обогащает исходное предложение контекстной информацией, а декодер формирует подробное представление предложения-перевода, используя данные (ключи и значения) от энкодера. Такая архитектура легла в основу всех больших языковых моделей (LLM).

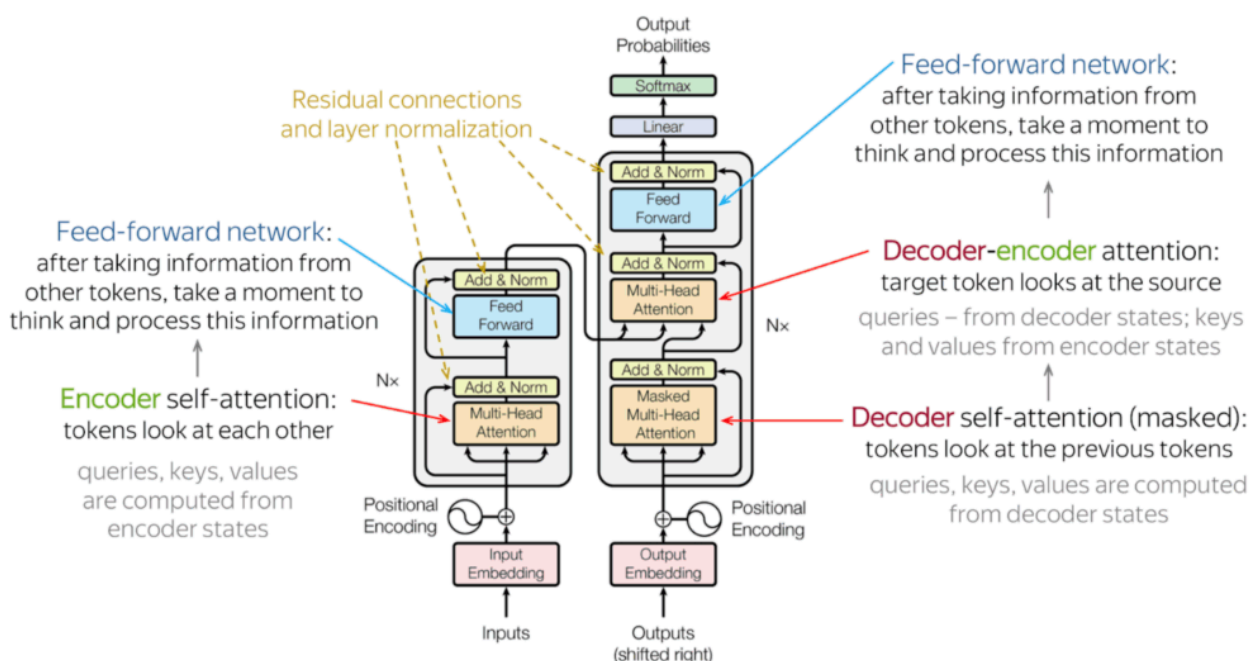


Рисунок 1. Архитектура трансформера [источник]

Типы моделей

Тип трансформера напрямую влияет на тип задач, с которыми модель может справиться. В зависимости от используемых компонентов выделяют три основные категории моделей:

- **Только энкодер** — модели строят репрезентативные эмбединги текста и хорошо подходят для задач классификации, извлечения информации, поиска. Пример — [BERT](#) (см. рисунок 2).

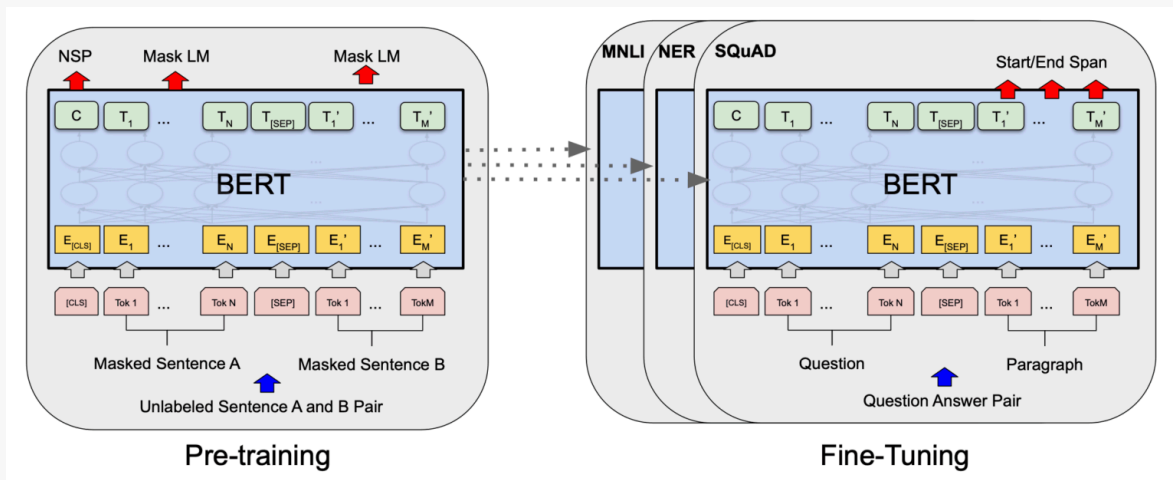


Рисунок 2. Общие процедуры предварительного обучения и fine-tuning'a для BERT. С (вверху слева) — выход модели на позиции токена [CLS], который используется как агрегированный эмбединг всего входного текста. Его часто применяют для классификации (например, определение отношения между двумя предложениями), предсказания следующего предложения. $T_1, T_2, \dots, T_N, T'_1, \dots, T'_M$ — это выходные эмбединги конкретных токенов [источник]

- **Только декодер** — модели генерируют последовательности (например, продолжения текста, написания ответов, креативных заданий и др.). Именно они лежат в основе современных LLM, способных к генеративным и интерактивным задачам. Пример — GPT (1, 2, 3), см. рисунок 3.

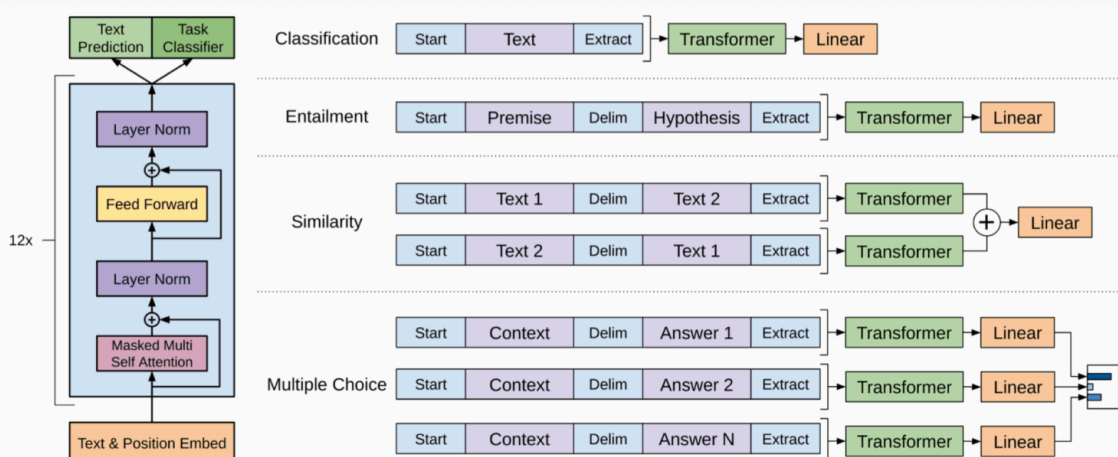


Рисунок 3. Модель GPT (2018): архитектура и цели обучения из оригинальной статьи (слева); входные преобразования для fine-tuning'a в различных задачах (справа) [источник]

- **Энкодер-декодер** — модели используются в машинном переводе, суммаризации и ответах на поставленные вопросы. Они одновременно создают представление входного текста (энкодером) и используют его при генерации нового текста (декодером). В некоторых случаях могут склоняться к шаблонным или менее разнообразным ответам. Пример — [T5](#) (см. рисунок 4).

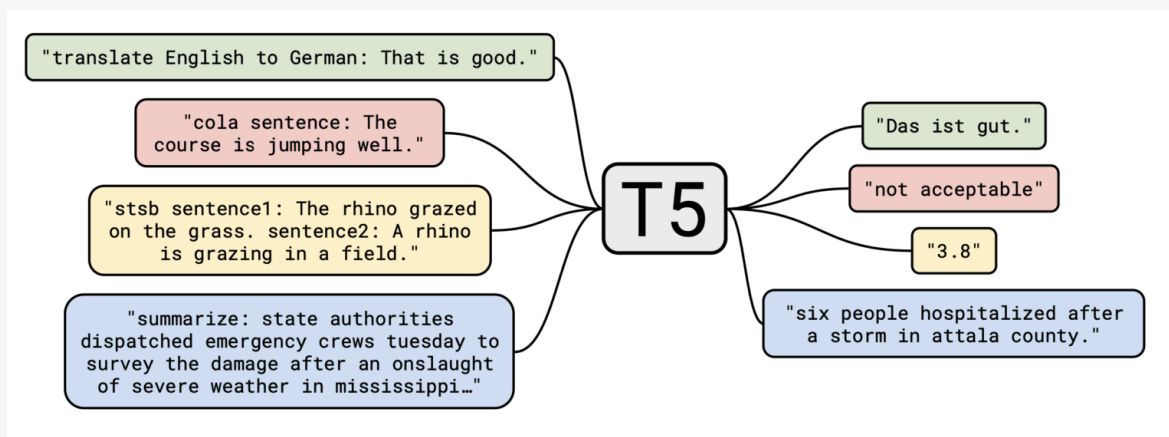


Рисунок 4. Схема структуры преобразования текста в текст модели T5. Каждая задача, включая перевод, ответы на вопросы и классификацию, считается подачей типового текста в качестве ввода и обучения для создания некоторого целевого текста [источник]

Предварительное обучение LLM

LLM обычно используются в виде предобученных (pre-trained) моделей. На этапе предварительного обучения они обрабатывают огромные объёмы немаркированного текста, что позволяет им:

1. Построить представление языка: как слова соотносятся с их значением (семантика), как строятся грамматические конструкции (синтаксис), как учитываются предыдущие слова и фразы при интерпретации текущих (контекст);
2. Получить начальные веса для последующего дообучения под конкретные задачи (fine-tuning);
3. Сформировать вероятностную модель языка, позволяющую сэмплировать токены на выходе: модель учится предсказывать следующий токен с вероятностной точки зрения, эта способность позволяет ей генерировать

текст, выбирая наиболее вероятные последовательности токенов или сэмплируя из распределения.

Одной из первых LLM, продемонстрировавших возможности zero-shot обучения, стала **GPT-2 (2019)**, обученная на 117 млн параметров. Уже тогда она могла решать задачи, на которых не была явно обучена.

Последующие модели (GPT-3, GPT-4 и др.) значительно увеличили масштаб (до сотен миллиардов параметров), процедура предобучения тоже усложнилась и стала многоэтапной. Это привело к улучшению качества генерации, но при этом модели остались текстовыми — без понимания изображений, аудио и других модальностей.

CLIP (OpenAI) [2021 г.]

Задача **CLIP** — корректно сопоставлять изображение и его текстовое описание. Модель обучалась на парах «изображение-текст». Принцип работы следующий:

1. Множество текстов (например, «pepper the aussie pup») подаются в текстовый энкодер.
2. Соответствующие текстам изображения подаются в отдельный энкодер изображений.
3. Оба энкодера преобразуют текст и изображения в векторные представления (Т и I), которые затем сравниваются.

Цель — минимизировать расстояние между векторами соответствующих пар и максимизировать между несоответствующими (рисунок 5). Такой подход называется **contrastive learning**. Благодаря CLIP появился поиск изображений по текстовому описанию (и наоборот).

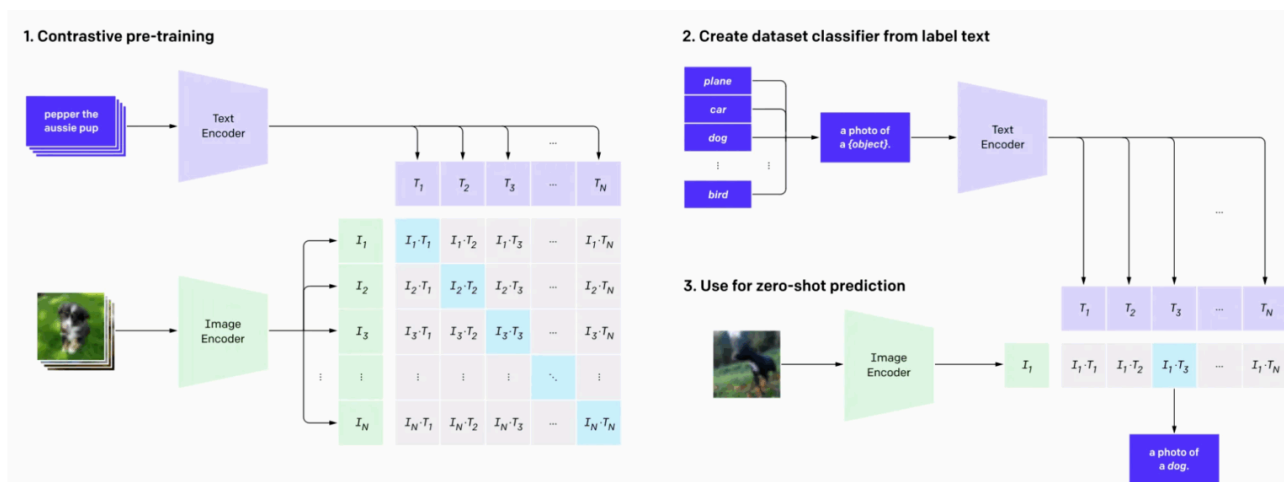


Рисунок 5. Модель CLIP [источник]

Flamingo (DeepMind) [2022 г.]

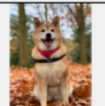
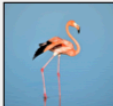
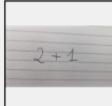
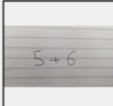
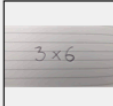
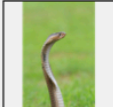
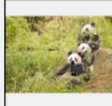
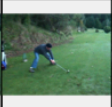
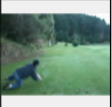
CLIP стал важным шагом для объединения изображений и текста — предложил эффективный способ сопоставления их представлений в общем эмбединговом пространстве с помощью двух независимых энкодеров и contrastive learning. Однако применимость CLIP ограничена задачами, требующими соотнесения модальностей: классификация и поиск (retrieval). Модель слабо адаптируется к новым мультимодальным задачам без дообучения.

Для решения ограничений была предложена мультимодальная модель [Flamingo](#), объединяющая визуальную информацию с мощной языковой моделью. Это достигается за счёт архитектурных механизмов cross-attention и адаптера (**perceiver**), которые позволяют обрабатывать изображения вместе с текстом и выполнять мультимодальные задачи: генерация описаний, ответы на вопросы по картинке и визуальное рассуждение.

Важно: Flamingo не требует полного переобучения языковой модели — вместо этого она подключается как модуль к уже **предобученной LLM**, превращая её в **VLM (Vision-Language Model)** с возможностью генерации и понимания мультимодальных данных. Ключевая особенность этой модели — **few-shot обучение (few-shot learning)**.

Flamingo обучается один раз на большом корпусе разнородных мультимодальных данных, которые включают пары «изображение + текст» из множества задач (описания изображений, QA по картинке, reasoning, см.

рисунок 6). Затем используется в few-shot режиме, как GPT-3: достаточно показать несколько примеров задачи прямо во входной последовательности, чтобы модель адаптировалась к новому типу задачи без обновления параметров.

Input Prompt					Completion
	This is a chinchilla. They are mainly found in Chile.		This is a shiba. They are very popular in Japan.		This is → a flamingo. They are found in the Caribbean and South America.
	What is the title of this painting? Answer: The Hallucinogenic Toreador.		Where is this painting displayed? Answer: Louvres Museum, Paris.		What is the name of the city where this was painted? Answer: → Arles.
	Output: "Underground"		Output: "Congress"		Output: → "Souloumes"
	2+1=3		5+6=11		 → 3x6=18
	Output: A propaganda poster depicting a cat dressed as French emperor Napoleon holding a piece of cheese.		Output: A pink room with a flamingo pool float.		Output: → A portrait of Salvador Dali with a robot head.
	Les sanglots longs des violons de l'automne blessent mon cœur d'une langueur monotone.		Pour qui sont ces serpents qui sifflent sur vos têtes?		 → Je suis un cœur qui bat pour vous.
	pandas: 3		dogs: 2		 → giraffes: 4
I like reading		, my favourite play is Hamlet. I also like		, my favorite book is	 → Dreams from my Father.
					What happens to the man after hitting the ball? Answer: → he falls down.

			
This is a picture of two teddy bears on the moon. What are they doing? They are having a conversation. What object are they using? It looks like a computer. Is this surprising? Yes, it is surprising. Why is this picture surprising to you? I think it is surprising because teddy bears are not usually found on the moon.	What is the common thing about these three images? They are all flamingos. What is the difference between these three images? The first one is a cartoon, the second one is a real flamingo, and the third one is a 3D model of a flamingo.	This is an apple with a sticker on it. What does the sticker say? The sticker says "iPod". Where is the photo taken? It looks like it's taken in a backyard. Do you think it is printed or handwritten? It looks like it's handwritten. What color is the sticker? It's white.	This is a cityscape. It looks like Chicago. What makes you think this is Chicago? I think it's Chicago because of the Shedd Aquarium in the background. What about this one? Which city is this and what famous landmark helped you recognise the city? This is Tokyo. I think it's Tokyo because of the Tokyo Tower.

Рисунок 6. Избранные примеры входных и выходных данных, полученных от Flamingo-80B. Flamingo может быстро адаптироваться к различным задачам через понимание

изображений / видео с помощью нескольких снимков (вверху). Flamingo способна к многообразному визуальному диалогу (внизу) [источник]

В Flamingo реализованы три ключевых архитектурных элемента (рисунок 7, 8, 9):

1. **Patch-tokenization изображений** — изображение разбивается на части (патчи, например, размером 16×16) и кодируется с помощью ViT (Vision Transformer), как и текст разбивается на токены. Эта последовательность визуальных токенов поступает в мультимодальный модуль.

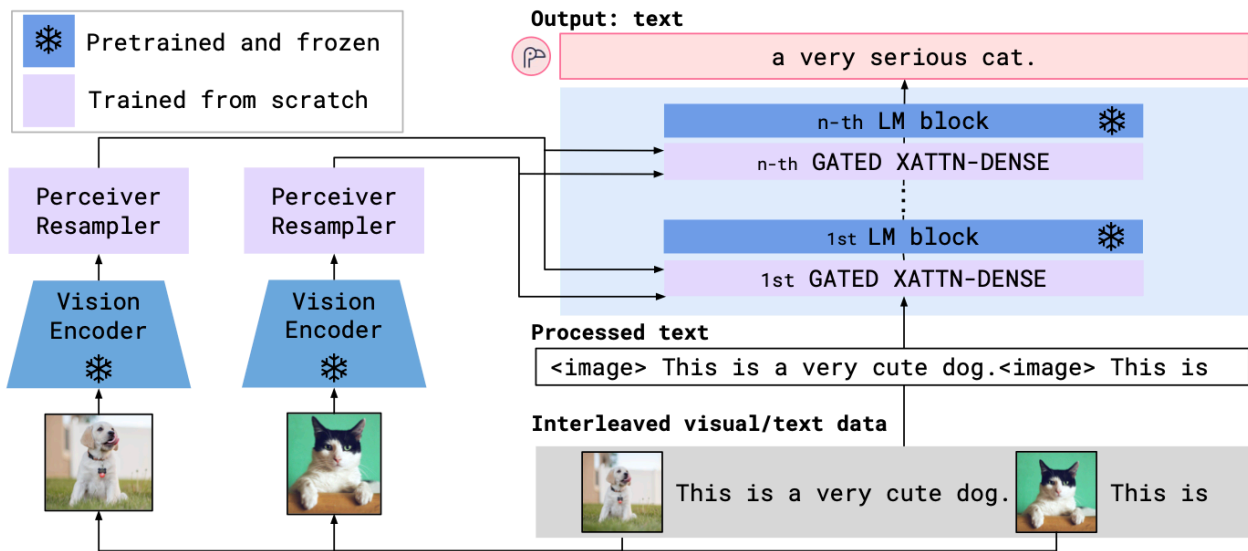


Рисунок 7. Обзор архитектуры Flamingo. Flamingo — семейство визуальных языковых моделей (VLM), которые берут в качестве входных данных визуальные данные, перемешанные с текстом, и производят текст свободной формы в качестве вывода [источник]

2. **Cross-attention** (рис. 7, 8) — особый механизм внимания, принимающий на вход как текстовые, так и визуальные эмбединги. На этапе генерации каждый слой LLM может «смотреть» на визуальные признаки через перекрёстное внимание по ключам и значениям из визуальных токенов.

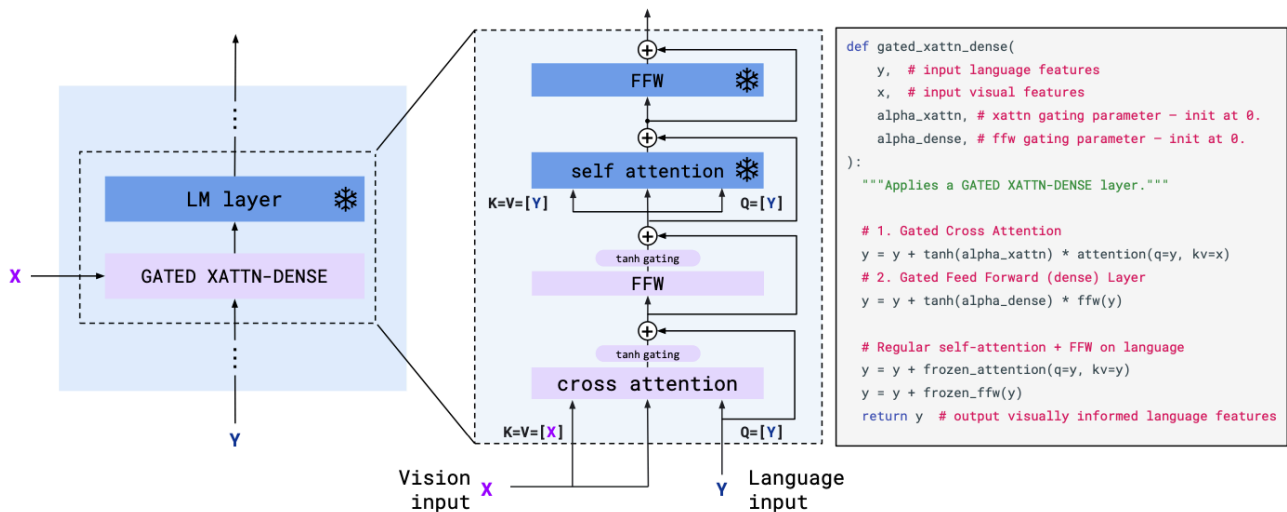


Рисунок 8. Слой GATED XATTN-DENSE. Для LM на визуальных входах Flamingo использует новые слои перекрёстного внимания между предварительно обученными и замороженными слоями LM. Ключи и значения в этих слоях получены из признаков зрения, в то время как запросы получены из языковых входов. За ними следуют feed-forward слои. Они «гейтированы», так что LM остаётся нетронутой при инициализации для улучшения стабильности и производительности [источник]

3. **Perceiver Resampler** (рис. 7, 9) — модуль, принимающий ViT-эмбединги и возвращающий фиксированное число обобщённых визуальных токенов, сокращает количество визуальных токенов и не перегружает LLM. Модуль уменьшает вычислительную сложность и унифицирует визуальное представление независимо от размера входного изображения.

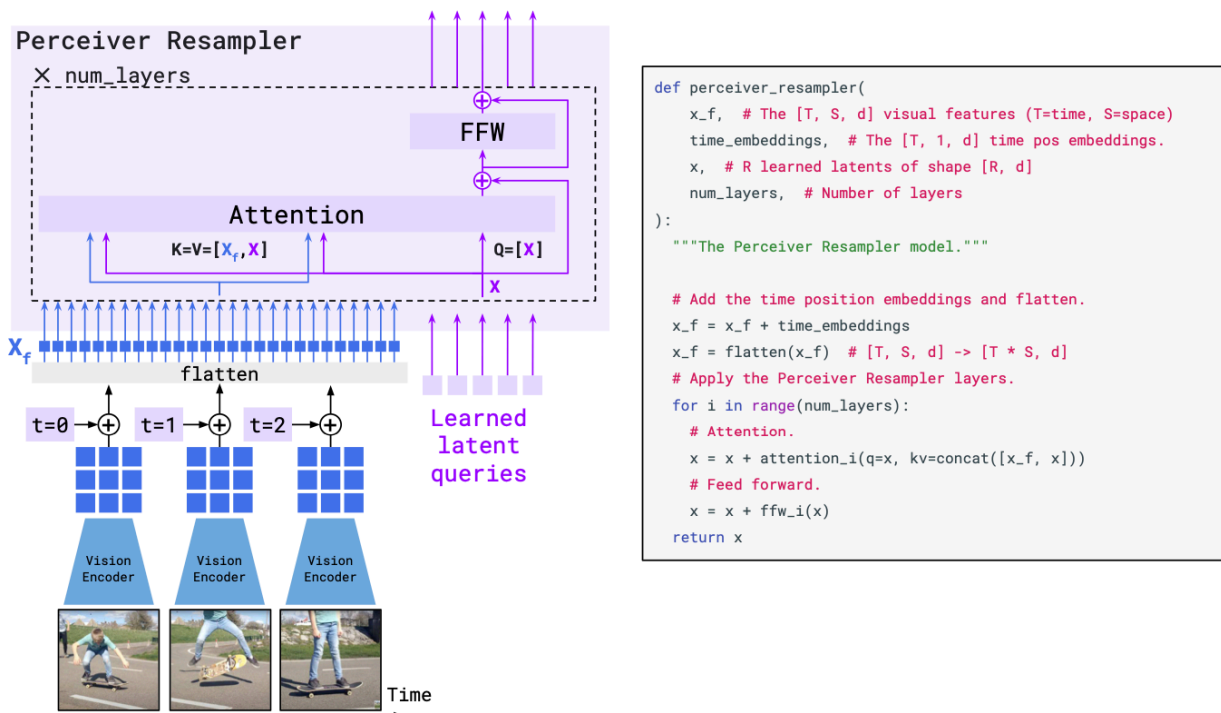


Рисунок 9. Perceiver Resampler отображает сетку переменного размера пространственно-временных визуальных признаков, выведенных Vision Encoder, на фиксированное количество выходных токенов (на рисунке их 5), независимо от

разрешения входного изображения или количества входных видеокадров. Трансформер имеет набор изученных скрытых векторов в виде запросов, а ключи и значения — конкатенация пространственно-временных визуальных признаков с изученными латентными векторами [\[источник\]](#)

Flamingo работает с чередующимися последовательностями вида:

$[Image_1, Text_1, Image_2, Text_2, \dots, Image_n, Text_n]$

Эта последовательность обрабатывается как единый вход в авторегрессионном режиме. На выходе VLM генерирует текст, используя как текущий текстовый контекст, так и мультимодальные ключи / значения из перекрёстного внимания на изображениях.

Именно Flamingo впервые позволил VLM не просто понимать визуальный контекст, но и включать его в полноценную цепочку рассуждений.

3D-LLM и пространственное мышление

Ещё одна важная веха в развитии мультимодальных языковых моделей — переход от понимания 2D-изображений к **трёхмерному восприятию мира**. Мы живём в объёмной среде, где объекты взаимодействуют, перемещаются и изменяются во времени. Для создания ИИ, способного понимать и действовать в реальности, модели должны уметь интерпретировать **трёхмерные сцены и структуры**.

Модели нового поколения, как **3D-LLM** и **Uni3D**, работают с различными форматами 3D-данных: облаками точек, вокселями, мешами и сценами, полученными с помощью LiDAR-сканеров, CAD-моделей и AR/VR-устройств.

3D-LLM [2023 г.]

[3D-LLM](#) (см. рисунок 10) фокусируется на объединении визуального 3D-входа и языкового выхода. Она может описывать объекты, определять их стиль, материал и функцию. Например, модель распознаёт, что перед ней кресло в стиле модерн из дерева, и может сгенерировать инструкции по сборке или предложения по редизайну (рисунок 11).

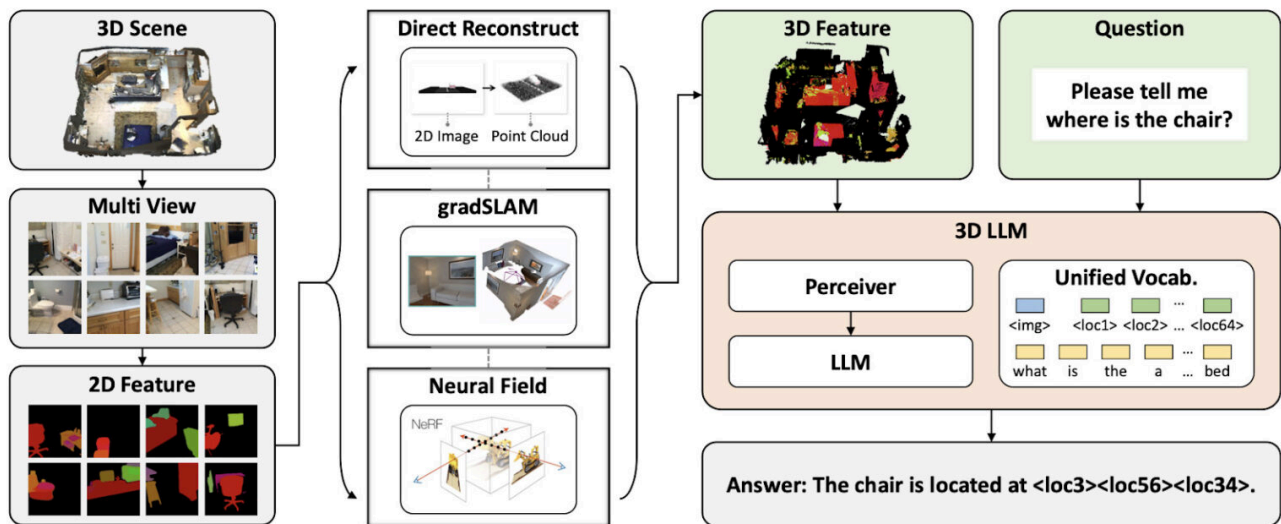


Рисунок 10. Обзор архитектуры 3D-LLM. Первые два столбца — извлечение признаков из 3D-сцены. Сначала происходит визуализация нескольких изображений сцены с разных точек зрения, извлечение 2D-признаков, а затем формируются 3D-признаки из этих мультивидовых изображений с помощью одного из трёх методов. Далее 3D-признаки вместе с текстовыми запросами подаются в 3D-LLM для генерации ответов [источник]

Архитектура 3D-LLM состоит из трёх ключевых компонентов:

1. Извлечение 3D-признаков (3D Feature Extractor)

3D-признаки извлекаются **не напрямую**, а из **2D-изображений**, **сгенерированных рендерингом 3D-сцены с разных ракурсов**. То есть, у нас уже есть **3D-сцена** (например, в виде 3D-модели, облака точек или неявного представления), и мы виртуально «фотографируем» её с разных ракурсов, чтобы получить 2D-изображения:

- используется предобученный 2D-энкодер (например, CLIP, BLIP);
- признаки с изображений объединяются в пространстве с помощью одного из трёх методов:
 - **Direct Reconstruction** — восстановление 3D-точек из RGBD-изображений и сопоставление с признаками;
 - **Feature Fusion (gradSLAM)** — 2D-признаки из изображений проецируются в 3D-пространство, формируя **трёхмерную карту признаков**;
 - **Neural Field** — кодирование пространства через воксельное нейросетевое представление, применимое при отсутствии глубины и шумных позах.

Результат — массив размерности $\langle N, D_v \rangle$, где N — количество точек, D_v — размерность признака (например, 1408 или 1024).

2. Подключение к LLM через 2D VLM (BLIP-2, Flamingo и др.)

Чтобы не обучать LLM с нуля, используются **предобученные визуально-языковые модели**, куда вводятся 3D-признаки:

- **3D-признаки**, извлечённые с помощью 3D Feature Extractor, проецируются в тот же признаковый (эмбединг) простор, что и признаки 2D-изображений, полученные, например, с помощью CLIP;
- **перцептроноподобный модуль (Perceiver/QFormer)** применяется для сжатия признаков и передачи в LLM;
- LLM работает по стандартной схеме: на вход подаются текстовые промпты и признаки сцены, на выходе — генерация текста (ответы, описания, инструкции).

3. 3D-локализация (3D Localization Mechanism)

Чтобы LLM могла «понимать» пространство, используется:

- **позиционное кодирование**: к каждому 3D-признаку добавляются синусо-косинусные эмбединги координат (x , y , z);
- **дискретизация координат** и добавление специальных **location tokens** в словарь LLM, позволяющих указывать пространственные позиции напрямую в языке;
- веса самой языковой модели остаются замороженными, но веса эмбедингов location tokens размораживаются и дообучаются, позволяя LLM ассоциировать текст с конкретными областями сцены (например, «под столом» → нижний регион bounding box).



Рисунок 11. Качественные примеры предсказания 3D-LLM [источник]

Uni3D [2023 г.]

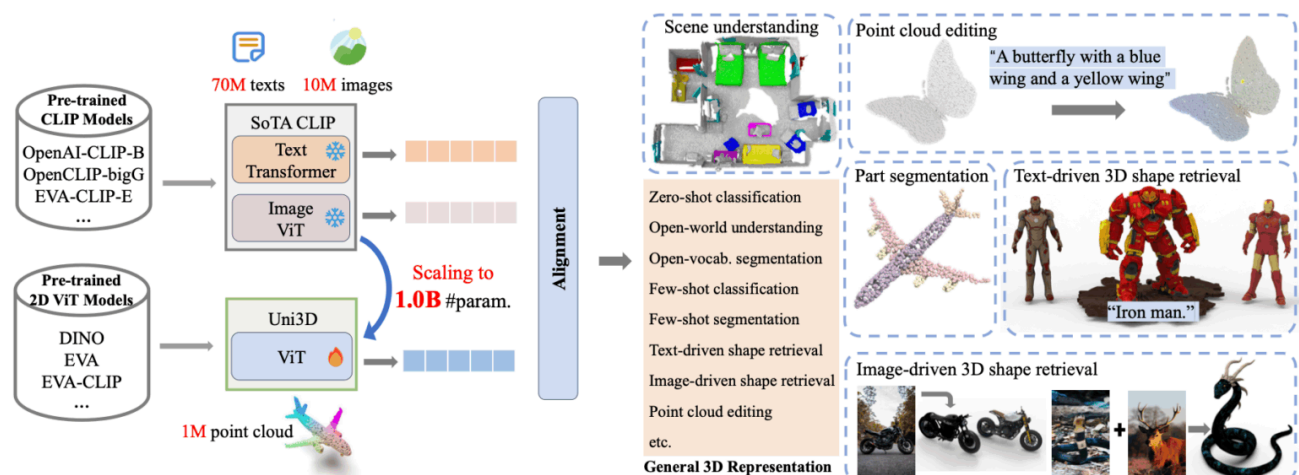
Один из главных вызовов в работе с трёхмерными данными — отсутствие универсальных моделей, способных решать широкий спектр задач без необходимости настройки под каждую из них. Модель [Uni3D](#) (см. рисунок 12) предлагает решение этой проблемы и вводит масштабируемую архитектуру, которая объединяет 3D, изображения и текст в единое мультимодальное пространство признаков.

Ключевая идея Uni3D — единое 3D-представление, совместимое с CLIP-пространством. Это открывает возможности для взаимодействия трёх модальностей (3D–текст–изображение) без перекодирования и дополнительных интерфейсов, что делает Uni3D мощной основой для интеграции с LLM и агентными системами.

В основе Uni3D лежит классическая архитектура Vision Transformer (ViT), адаптированная под 3D-вход. Вместо 2D-патчей здесь используется **облачный токенизатор** на основе **PointBERT**: облако точек разбивается на локальные регионы, каждый из которых кодируется компактным признаковым вектором. Эти токены затем подаются в ViT, инициализированный весами из 2D-модели. Такая архитектурная унификация позволяет эффективно переносить знания из визуально-текстовых моделей в 3D-домен.

Uni3D обучается с помощью contrastive triplet learning, выравнивая представления облаков точек с изображениями и текстовыми описаниями. На вход модели подаются триплеты: 3D-объект, изображение, описание. Изображения и тексты проходят через замороженные CLIP-энкодеры, а 3D-энкодер адаптируется, чтобы его выходные представления занимали **общее семантическое пространство** с другими модальностями.

Благодаря такому обучению Uni3D способна работать в zero-shot (генерация по запросу без примеров) и few-shot (генерация по запросу с несколькими примерами) режимах на множестве задач: классификация форм, сегментация объектов, поиск по описанию, 3D-captioning. При этом модель масштабируется вплоть до миллиарда параметров без изменения архитектуры, а ViT-структура позволяет использовать многообразие подходов из NLP и 2D-зрения (например, позиционное кодирование, визуальные промпты и fine-tuning).



Заключение

Итак, мы проследили путь от классических языковых моделей, работающих только с текстом, к современным системам, способным **воспринимать и интерпретировать** 2D и 3D-сцены. Эти изменения открывают новую эру в развитии искусственного интеллекта: модели больше не ограничены символьной информацией, они начинают **«видеть»** — распознавать объекты, сцены, действия, контекст.

Мультимодальность в визуальном измерении — не просто добавление картинок к тексту, а переход к системам, которые могут **понимать** окружающий мир через зрительное восприятие. Это делает возможным появление ИИ-систем, способных объяснять, рассуждать и взаимодействовать с визуальной информацией на уровне, приближенном к человеческому.

Такие технологии лежат в основе новых приложений: от ассистентов, умеющих интерпретировать фотографии и графики, до научных и медицинских систем, анализирующих визуальные данные. И в центре этих изменений — **языковые модели**, которые становятся ядром, объединяющим текст и изображение в общее пространство смысла.

Понимание принципов работы LLM — основа, без которой невозможно осваивать современные визуальные ИИ-системы. Ведь именно языковые модели служат центральным связующим элементом, объединяющим различные модальности в единое пространство понимания. Качественную базу можно заложить у нас на курсе по LLM.



Поделиться

